

KompozeR

Applicazioni e Servizi Web

Valerio Giannini - 0001125005 {valerio.giannini@studio.unibo.it}

xx giugno 2026

0.1 Introduzione

KompozeR è una web application che estende le funzionalità dell'e-commerce di Kompo (www.soluzionekompo.com), azienda specializzata in mobili componibili personalizzabili. L'applicazione permette all'utente di progettare la propria scaffalatura tramite un configuratore visuale interattivo, ottenendo un'anteprima del risultato finale e popolando automaticamente il carrello con i componenti selezionati. L'utente oltre alla creazione del proprio scaffale personalizzato potrà accedere a funzionalità avanzate di assistenza e monitoraggio in tempo reale. Un chatbot assistente in tempo reale guida l'utente durante la configurazione, rispondendo a domande su dimensioni, compatibilità e prezzi. Il sistema notifica inoltre l'utente in tempo reale in caso di variazioni di disponibilità o prezzo dei componenti inseriti nella propria configurazione. Lato amministratore, KompozeR fornisce strumenti per la gestione del catalogo prodotti, un sistema di monitoraggio degli ordini (che traccia l'arrivo e la spedizione dei componenti) e una pagina di reportistica per vedere i trend di vendita.

0.1.1 Contesto

Kompo è un'azienda che si occupa di mobili componibili personalizzabili, offrendo soluzioni su misura per ogni esigenza abitativa. Il catalogo si divide in 3 categorie di prodotti: Tondo, Quadro e Qube; non compatibili tra loro, ma con caratteristiche e funzionalità simili (Questo dettaglio è importante per definire in seguito la logica di componimento dei singoli componenti: che è la medesima per ogni tipologia di prodotto). Il processo di acquisto attuale prevede che l'utente selezioni i singoli componenti desiderati, avendo già un'idea chiara del risultato finale, e li aggiunga al carrello. Questo processo può risultare complesso e poco intuitivo per gli utenti che non conoscono il prodotto e si trovano costretti a chiedere assistenza direttamente al personale di vendita, con conseguente aumento dei tempi di acquisto e potenziale insoddisfazione del cliente. KompozeR nasce con l'obiettivo di semplificare e migliorare l'esperienza di acquisto, offrendo un configuratore visuale il più semplice e intuitivo possibile, garantendo a tutti la possibilità di acquisto.

0.2 Requisiti

La definizione dei requisiti di KompozeR è stata costruita seguendo un approccio orientato all'utente. In particolare, la fase di analisi è stata articolata a partire dall'individuazione degli stakeholder principali, dalla costruzione di personas rappresentative e dalla descrizione di scenari e casi d'uso centrali. Questo consente di giustificare i requisiti di business, funzionali e non funzionali come derivazione diretta di esigenze concrete del dominio applicativo.

0.2.1 Stakeholder

L'identificazione degli stakeholder consente di chiarire chi trae valore dal sistema, chi lo utilizza direttamente e chi influenza i vincoli progettuali.

Stakeholder primari

Stakeholder	Descrizione	Obiettivi principali	Bisogni critici
Cliente finale	Utente che desidera progettare e acquistare una scaffalatura personalizzata	Configurare rapidamente il prodotto corretto, capire prezzo e compatibilita', completare l'acquisto senza errori	Configuratore chiaro, anteprima affidabile, salvataggio configurazioni
Cliente registrato ricorrente	Utente che torna sul sito per modificare o riordinare una configurazione	Riutilizzare o modificare configurazioni salvate	Storico configurazioni, accesso rapido
Amministratore catalogo	Operatore che gestisce prodotti, componenti, disponibilita' e prezzi	Mantenere coerente il catalogo e rendere acquistabili solo combinazioni valide	Interfacce di gestione chiare, aggiornamenti rapidi, controllo consistenza dati

Stakeholder secondari

Stakeholder	Ruolo nel progetto	Interesse principale
Azienda Kompo	Committente del prodotto	Aumentare conversione, ridurre abbandono, migliorare esperienza d'acquisto
Team di sviluppo	Realizza e mantiene il sistema	Requisiti chiari, comportamento verificabile, architettura evolvibile
Servizio clienti	Supporta utenti con dubbi o problemi	Ridurre richieste ripetitive grazie a configuratore e assistenza automatizzata
Responsabile business/reportistica	Analizza andamento vendite e utilizzo del sistema	Ottenere insight utili su trend di vendita e utilizzo delle funzionalita'

0.2.2 Personas

Per guidare la definizione dei requisiti sono state individuate tre personas rappresentative.

Marta, cliente orientata alla praticita'

Marta ha 34 anni, e' architetta freelance e desidera arredare uno studio domestico con una soluzione modulare compatibile con lo spazio disponibile. Ha buone

competenze digitali e si aspetta un configuratore guidato, capace di mostrare chiaramente il risultato finale e il prezzo complessivo. Le sue principali frustrazioni riguardano la complessità dei cataloghi tecnici, i dubbi sulla compatibilità tra componenti e il rischio di perdere il lavoro svolto durante la configurazione.

Luca, cliente indeciso ma motivato all'acquisto

Luca ha 28 anni, è impiegato e vuole arredare il soggiorno pur non conoscendo in dettaglio il dominio dei mobili componibili. Ha competenze digitali medie e necessita di un'interfaccia semplice, di un supporto contestuale sui prezzi e della possibilità di salvare una configurazione per riprenderla in seguito. La sua esigenza principale è evitare errori di comprensione durante il processo di acquisto.

Giulia, amministratrice del catalogo

Giulia ha 41 anni ed è responsabile operativa dell'e-commerce. Si occupa di aggiornare prezzi e disponibilità dei componenti e ha bisogno di strumenti amministrativi chiari, rapidi e coerenti con il resto del sistema. Il suo obiettivo è garantire che le configurazioni costruite dagli utenti restino allineate con i dati reali del catalogo e con le informazioni operative.

0.2.3 Scenari d'uso principali

Gli scenari d'uso permettono di rappresentare in modo narrativo le interazioni rilevanti da cui derivano i requisiti del sistema.

- **Configurazione iniziale:** l'utente seleziona una categoria di prodotto, inserisce eventuali preferenze iniziali e compone la scaffalatura attraverso il configuratore visuale. Il sistema mostra soltanto componenti compatibili e aggiorna dinamicamente anteprima e prezzo.
- **Supporto contestuale:** durante la configurazione l'utente apre il chatbot e richiede chiarimenti sul prezzo di uno o più componenti, senza interrompere il flusso operativo principale.
- **Salvataggio e ripresa:** l'utente autenticato salva una configurazione, la recupera in una sessione successiva e la modifica prima dell'acquisto.
- **Notifica di variazione:** il sistema informa l'utente quando una configurazione salvata o attiva viene impattata da una variazione di prezzo o disponibilità di uno dei componenti coinvolti.
- **Gestione amministrativa:** l'amministratore aggiorna dati di catalogo e consulta ordini e report sintetici per monitorare l'andamento operativo e commerciale.

0.2.4 Casi d'uso principali

I casi d'uso principali del sistema sono riportati di seguito in forma sintetica.

UC1 – Configurazione di una scaffalatura

Attore principale Cliente finale.

Precondizioni Il configuratore e' disponibile e il sistema dispone dei componenti necessari alla composizione.

Flusso principale L'utente avvia il configuratore, seleziona o conferma la categoria di prodotto, inserisce eventuali vincoli iniziali, modifica la configurazione e visualizza gli aggiornamenti dinamici di anteprima e prezzo. Il sistema consente esclusivamente l'uso di componenti compatibili con la categoria scelta.

Postcondizioni Esiste una configurazione coerente, pronta per essere salvata o aggiunta al carrello.

UC2 – Richiesta di assistenza contestuale

Attore principale Cliente finale.

Precondizioni L'utente si trova in una fase di configurazione e il chatbot e' disponibile.

Flusso principale L'utente apre il chatbot, formula una domanda sul prezzo di uno o piu' componenti e riceve una risposta contestuale che gli consente di proseguire la configurazione.

Postcondizioni L'utente ottiene supporto senza uscire dal configuratore.

UC3 – Salvataggio e ripresa di una configurazione

Attore principale Utente autenticato.

Precondizioni L'utente ha creato almeno una configurazione ed e' autenticato.

Flusso principale L'utente salva la configurazione corrente, la ritrova nel proprio profilo in una sessione successiva e la riapre per modificarla o completarla.

Postcondizioni La configurazione risulta persistita e riutilizzabile in sessioni future.

UC4 – Notifica di variazione prezzo o disponibilit 

Attore principale Utente autenticato.

Attore secondario Amministratore catalogo.

Precondizioni Esiste almeno una configurazione attiva o salvata e uno dei componenti associati subisce una variazione rilevante.

Flusso principale L'amministratore aggiorna il dato di catalogo, il sistema individua le configurazioni impattate e invia una notifica agli utenti coinvolti.

Postcondizioni L'utente   informato delle variazioni che possono influenzare la propria configurazione.

UC5 – Gestione amministrativa e reportistica

Attore principale Amministratore.

Precondizioni L'amministratore   autenticato nell'area gestionale.

Flusso principale L'amministratore aggiorna prezzi e disponibilit  dei componenti, consulta gli ordini e accede a una sezione di reportistica sintetica.

Postcondizioni I dati risultano aggiornati e l'amministratore dispone di informazioni utili al monitoraggio operativo e commerciale.

0.2.5 Requisiti di business

BR1 Il sistema deve supportare la vendita di scaffalature personalizzabili semplificando il processo di composizione rispetto alla procedura di acquisto tradizionale.

BR2 Il sistema deve aumentare il valore percepito dell'e-commerce Kompo offrendo un configuratore visuale e servizi di assistenza digitale integrati.

BR3 Il sistema deve ridurre il rischio di errori d'acquisto dovuti a scarsa comprensione di prezzi, disponibilit , compatibilit  tra componenti e stato della configurazione.

BR4 Il sistema deve consentire all'azienda di mantenere allineati configurazioni utente, dati di catalogo e informazioni operative.

BR5 Il sistema deve fornire all'area amministrativa una vista sintetica utile al monitoraggio di ordini e trend di vendita.

0.2.6 Requisiti funzionali

- FR1** Il sistema deve fornire un configuratore visuale per la composizione della scaffalatura all'interno della categoria di prodotto selezionata.
- FR2** Il sistema deve aggiornare l'anteprima della scaffalatura in seguito alle modifiche effettuate nel configuratore.
- FR3** Il sistema deve aggiornare il prezzo stimato della configurazione in seguito alle modifiche effettuate nel configuratore.
- FR4** Il sistema deve consentire di aggiungere al carrello una configurazione completata.
- FR5** Il sistema deve fornire un chatbot integrato per rispondere a richieste contestuali sui prezzi dei componenti.
- FR6** Il sistema deve consentire agli utenti autenticati di salvare una configurazione associandola al proprio profilo.
- FR7** Il sistema deve consentire agli utenti autenticati di visualizzare e riprendere configurazioni salvate in precedenza.
- FR8** Il sistema deve notificare agli utenti autenticati variazioni di prezzo o disponibilit  che impattano configurazioni attive o salvate.
- FR9** Il sistema deve impedire la composizione di scaffalature che mescolano componenti appartenenti a categorie di prodotto diverse.
- FR10** Il sistema deve consentire agli amministratori di aggiornare dati di catalogo, inclusi prezzo e disponibilit  dei componenti.
- FR11** Il sistema deve rendere disponibili agli amministratori una vista ordini e una sezione di reportistica sintetica sui trend di vendita.

0.2.7 Requisiti non funzionali

- NFR1** L'interfaccia del configuratore deve permettere a un utente con competenze digitali medie di comprendere le azioni principali senza formazione preventiva.
- NFR2** Il sistema deve mostrare feedback immediato a seguito delle azioni rilevanti dell'utente, in particolare durante configurazione, salvataggio e aggiornamento prezzo.
- NFR3** Le principali funzionalit  del sistema devono essere progettate in modo coerente con i principi base di accessibilit  web e con l'uso corretto di elementi semantici.
- NFR4** L'interfaccia deve risultare fruibile sia da dispositivi desktop sia da dispositivi mobili, preservando l'accesso alle funzionalit  essenziali.

NFR5 L'aggiornamento di anteprima e prezzo nel configuratore deve essere percepito dall'utente come tempestivo durante l'interazione.

NFR6 Le notifiche di variazione prezzo o disponibilit  devono essere recapitate in modo consistente agli utenti interessati quando il contesto applicativo lo consente.

NFR7 Le funzionalit  amministrative devono essere accessibili solo ad utenti autenticati con ruolo autorizzato.

NFR8 Il sistema deve memorizzare soltanto i dati necessari alla gestione di account, configurazioni salvate e operazioni applicative previste.

NFR9 Il sistema deve mantenere separati i moduli relativi a configurazione, gestione catalogo, notifiche e reportistica in modo da facilitare evoluzione e manutenzione.

NFR10 Il sistema deve evitare funzionalit  superflue e privilegiare interazioni essenziali, riducendo complessit , traffico e carico computazionale non necessario.

0.2.8 Tracciabilit 

I requisiti presentati in questa sezione derivano dall'analisi degli stakeholder, dalle personas individuate e dagli scenari e casi d'uso principali. Questa struttura garantisce la coerenza tra bisogni degli utenti, comportamento atteso del sistema e qualit  richieste al prodotto finale.

0.3 Design

Questa sezione descrive le principali scelte di progettazione del sistema dal punto di vista di Applicazioni e Servizi Web. L'obiettivo   spiegare come KompozeR sia stato organizzato come applicazione web moderna, chiarendo responsabilit  dei componenti, flussi di comunicazione, modello architetturale e organizzazione delle interfacce utente.

Gli aspetti piu' strettamente legati ai sistemi distribuiti, come coordinamento collaborativo avanzato, ordering causale, recovery e fault tolerance, sono richiamati solo quando necessari per comprendere le scelte architetturali generali e vengono trattati in dettaglio nel report dedicato a DS.

0.3.1 Obiettivi di design

Le principali decisioni progettuali sono state guidate dai seguenti obiettivi:

- separare in modo chiaro frontend, gateway e servizi backend;
- mantenere indipendenti i moduli principali del dominio, come autenticazione, catalogo, configurazione, carrello, notifiche e reporting;

- favorire manutenibilita', evoluzione incrementale e chiarezza delle responsabilita';
- supportare un'esperienza utente interattiva, con aggiornamenti rapidi di prezzo, configurazione e notifiche.

0.3.2 Visione architetturale

KompozeR e' stato progettato come una Single Page Application che comunica con un backend organizzato in piu' microservizi. Il frontend concentra la logica di presentazione e di interazione con l'utente, mentre il backend espone servizi applicativi separati per area di responsabilita'.

Dal punto di vista logico, l'architettura si articola in tre livelli principali:

- frontend web, responsabile dell'interazione con utenti finali e amministratori;
- API Gateway, che rappresenta il punto di ingresso unico per richieste HTTP e comunicazioni real-time;
- microservizi backend, ciascuno focalizzato su un sottoinsieme specifico del dominio.

In questa sottosezione e' opportuno inserire un diagramma architetturale generale che mostri SPA, gateway, servizi e database per servizio.

0.3.3 Suddivisione in microservizi

La scelta di suddividere il backend in microservizi nasce dall'esigenza di isolare responsabilita' diverse e contenere la complessita' del dominio.

I principali servizi individuati sono:

- `authenticationService`, responsabile di account, login, ruoli e sessioni;
- `catalogService`, responsabile dei prodotti, dei prezzi e della disponibilita';
- `cadService`, responsabile delle configurazioni create dagli utenti;
- `cartService`, responsabile del carrello e delle richieste d'ordine;
- `notificationService`, responsabile della gestione delle sottoscrizioni e delle notifiche utente;
- `chatbotService`, responsabile delle sessioni di assistenza conversazionale;
- `reportingService`, responsabile della generazione e consultazione dei report;
- `apiGateway`, responsabile dell'esposizione unificata delle API verso il frontend.

Per ciascun servizio e' consigliabile descrivere brevemente:

- responsabilita' principale;
- dati posseduti;
- principali interazioni con gli altri servizi.

0.3.4 API Gateway e comunicazione tra componenti

Il gateway consente di semplificare la comunicazione tra client e backend, centralizzando forwarding, validazione preliminare dei token e controllo degli accessi. In questo modo il frontend interagisce con un unico punto di ingresso, senza dover conoscere i dettagli interni dell'organizzazione dei servizi.

Le interazioni applicative principali avvengono tramite API REST, usate per autenticazione, gestione catalogo, operazioni sul carrello, recupero configurazioni e consultazione dei report. Alcune funzionalita' che richiedono aggiornamento tempestivo, come notifiche e assistenza conversazionale, fanno uso anche di comunicazione real-time.

In questa parte conviene sottolineare che il canale real-time e' rilevante anche per ASW come scelta di interazione web avanzata, mentre gli aspetti algoritmici e distribuiti piu' sofisticati vengono approfonditi separatamente nel report DS.

0.3.5 Organizzazione del frontend

Il frontend e' stato pensato come SPA con pagine e componenti organizzati per responsabilita' utente. Le principali viste applicative sono:

- area catalogo e accesso al configuratore;
- configuratore visuale della scaffalatura;
- area carrello e riepilogo richiesta d'ordine;
- area notifiche;
- interfaccia chatbot di supporto;
- area amministrativa per catalogo e reportistica.

Qui e' utile spiegare come il frontend mantenga separati stato di sessione, stato del catalogo, stato delle configurazioni e dati amministrativi, cosi' da rendere piu' chiara la manutenzione del codice e la coerenza delle interazioni.

0.3.6 Modello del dominio rilevante per ASW

Dal punto di vista applicativo, il dominio ruota attorno ad alcune entita' centrali: utente, prodotto, configurazione, carrello, richiesta d'ordine, sessione di chat e notifica. La progettazione ha cercato di mantenere chiaro il confine tra dati globali condivisi e concetti locali dei singoli servizi.

In questa sottosezione e' opportuno sintetizzare:

- le entita' principali del dominio;
- i bounded context principali;
- le relazioni tra configurazione, catalogo e carrello;
- il ruolo di utenti registrati, guest e amministratori.

Se necessario, si puo' inserire un diagramma concettuale semplificato, evitando pero' di appesantire il report con dettagli che saranno gia' sviluppati nel report DS.

0.3.7 Sicurezza e controllo accessi

La sicurezza applicativa si basa su autenticazione tramite token e su una distinzione chiara tra livelli di accesso. Il sistema prevede infatti operazioni pubbliche, operazioni accessibili a utenti autenticati e operazioni riservate agli amministratori.

In questa parte conviene spiegare:

- il ruolo del servizio di autenticazione;
- l'uso dei token per identificare l'utente lato backend;
- la distinzione tra utente guest, utente autenticato e amministratore;
- il fatto che il gateway rappresenta il primo punto di enforcement delle policy di accesso.

0.3.8 Design delle interfacce utente

Il design delle interfacce e' stato orientato a ridurre la complessita' percepita durante la composizione della scaffalatura. Le principali scelte sono state guidate dall'esigenza di mostrare solo le azioni rilevanti, mantenere visibili prezzo e stato della configurazione e fornire supporto contestuale senza interrompere il flusso principale.

Questa sottosezione puo' includere:

- mockup o screenshot delle viste principali;
- motivazioni di usabilita' per il configuratore;
- organizzazione dell'area amministrativa;
- modalita' di visualizzazione di notifiche e chatbot.

0.3.9 Considerazioni finali di progetto

Nel complesso, il design di KompozeR cerca di bilanciare semplicità di utilizzo lato utente e modularità lato backend. La struttura adottata consente di raccontare il progetto in modo coerente nel contesto ASW, concentrandosi su architettura web, progettazione dei servizi, interfacce e organizzazione applicativa, lasciando al report DS l'approfondimento delle dinamiche distribuite più specialistiche.

0.4 Tecnologie

Questa sezione descrive le principali tecnologie adottate nel progetto e le motivazioni che hanno guidato la loro selezione. L'obiettivo non è fornire un semplice elenco di strumenti, ma mostrare come ogni scelta sia coerente con i requisiti del sistema e con l'architettura descritta nella sezione precedente.

0.4.1 Stack frontend

Il frontend di KompozeR è stato concepito come una Single Page Application moderna, orientata a interazioni rapide e aggiornamento dinamico dello stato. In questa sottosezione è opportuno descrivere le tecnologie utilizzate per:

- rendering dell'interfaccia;
- gestione dello stato client;
- routing tra pagine e viste;
- integrazione con API REST e canali real-time.

Per ciascuna tecnologia selezionata conviene motivare la scelta in termini di produttività, chiarezza architetturale, ecosistema e aderenza ai requisiti dell'applicazione.

0.4.2 Stack backend

Il backend è organizzato come insieme di servizi web separati. Qui è utile descrivere:

- runtime applicativo;
- framework HTTP utilizzato per l'esposizione delle API;
- linguaggio e supporto alla tipizzazione;
- eventuale struttura interna comune dei microservizi.

In questa parte va spiegato perché una soluzione basata su TypeScript e servizi Express risulta adeguata per un sistema web modulare e facilmente evolvibile.

0.4.3 Persistenza dei dati

La persistenza e' organizzata secondo il principio di database per servizio. Questa sottosezione dovrebbe spiegare:

- la tecnologia di database scelta;
- le motivazioni di flessibilita' e velocita' di sviluppo;
- la separazione dei dati tra i diversi microservizi;
- l'allineamento tra modello logico del dominio e struttura dei documenti persistiti.

0.4.4 Comunicazione real-time

Alcune funzionalita' richiedono uno scambio di informazioni piu' immediato rispetto al classico modello request-response. In questa sottosezione e' utile presentare la tecnologia usata per la comunicazione real-time e spiegare in quali punti del sistema viene impiegata, ad esempio per notifiche utente e chatbot.

Convieni mantenere qui il focus sul valore applicativo e sull'esperienza utente, senza entrare nel dettaglio dei meccanismi distribuiti avanzati che saranno trattati nel report DS.

0.4.5 Containerizzazione e strumenti di esecuzione

Per il deployment locale e per l'avvio coordinato dei servizi e' utile descrivere gli strumenti utilizzati per containerizzazione e orchestrazione leggera. In particolare si possono motivare:

- uso di Docker per isolare i componenti;
- uso di `docker-compose` per l'avvio dell'intero sistema;
- vantaggi in termini di replicabilita' dell'ambiente e semplicita' di setup.

0.4.6 Tecnologie per test e sviluppo

Se previste o gia' utilizzate, questa sottosezione puo' descrivere le tecnologie di supporto allo sviluppo e alla qualita' del codice, ad esempio:

- strumenti di test unitario e di integrazione;
- linters e formatter;
- strumenti per debugging e ispezione delle API.

0.4.7 Valutazione complessiva dello stack

In chiusura, conviene sintetizzare perche' lo stack scelto sia adeguato al progetto: buona separazione delle responsabilita', facilita' di sviluppo incrementale, coerenza con una web application moderna e supporto alla futura evoluzione del sistema.

0.5 Codice

Questa sezione non ha l'obiettivo di documentare ogni dettaglio implementativo, ma di mostrare gli aspetti di codice piu' significativi per comprendere come le scelte architetturali siano state tradotte in una soluzione concreta.

0.5.1 Organizzazione della repository

In questa sottosezione e' utile descrivere la struttura generale del progetto, distinguendo tra frontend, backend e documentazione di supporto. Conviene mettere in evidenza come l'organizzazione delle cartelle rifletta la separazione dei moduli applicativi e favorisca manutenzione e comprensione del sistema.

Si puo' inserire un breve tree della repository o una descrizione sintetica delle directory principali.

0.5.2 Struttura tipo di un microservizio

Ogni microservizio del backend segue una struttura interna coerente, pensata per distinguere chiaramente logica di dominio, logica applicativa e componenti di integrazione tecnica. In questa parte conviene presentare la struttura tipo di un servizio, spiegando il ruolo di cartelle e moduli principali.

Questa sottosezione e' particolarmente utile per mostrare come il progetto mantenga ordine e coerenza interna anche in presenza di piu' servizi distinti.

0.5.3 Esempi di flussi implementativi rilevanti

Per evitare una descrizione dispersiva del codice, e' consigliabile selezionare pochi flussi rappresentativi e commentarli in modo mirato. Alcuni candidati naturali sono:

- autenticazione utente e gestione della sessione;
- creazione o salvataggio di una configurazione;
- aggiornamento del catalogo da parte di un amministratore;
- generazione di una notifica a seguito di una modifica rilevante.

Per ciascun flusso si possono mostrare uno o due snippet significativi, spiegando:

- punto di ingresso della richiesta;
- logica applicativa principale;
- componenti coinvolti;
- formato della risposta o effetto osservabile lato utente.

0.5.4 Componenti frontend significativi

Il frontend rappresenta una parte essenziale del progetto, poiche' ospita il configuratore, la navigazione dell'utente e l'interazione con i servizi backend. In questa sottosezione conviene illustrare i componenti piu' rilevanti, ad esempio:

- vista del configuratore;
- componenti di riepilogo prezzo e carrello;
- gestione dello stato di autenticazione;
- componenti per notifiche e chatbot.

L'obiettivo non e' mostrare l'intera implementazione dell'interfaccia, ma evidenziare i punti in cui la struttura del codice supporta in modo chiaro le esigenze applicative.

0.5.5 Contratti applicativi e validazione

Una parte rilevante del progetto riguarda la definizione chiara dei contratti dati tra frontend e backend. In questa sezione si puo' richiamare il lavoro svolto su payload REST, messaggi WebSocket e DTO logici, spiegando come questi contratti contribuiscano alla coerenza dell'implementazione.

Questa sottosezione e' utile anche per mostrare che il progetto e' stato costruito con attenzione alla stabilita' delle API e alla manutenibilita' del codice.

0.5.6 Considerazioni sull'implementazione

In chiusura, conviene sottolineare come l'implementazione rispecchi le decisioni di design: separazione dei moduli, centralita' dei contratti, chiarezza dei servizi e attenzione alla leggibilita' del codice. Anche in presenza di una base ancora in evoluzione, questa sezione deve far emergere la coerenza complessiva della soluzione realizzata.

0.6 Test

Questa sezione descrive la strategia di verifica del progetto, con l'obiettivo di mostrare come il sistema sia stato controllato dal punto di vista funzionale, applicativo e, dove possibile, dell'esperienza utente.

0.6.1 Obiettivi della fase di test

La fase di test ha lo scopo di verificare:

- correttezza dei flussi applicativi principali;
- coerenza delle API esposte dai servizi;
- corretto comportamento dell'interfaccia nei casi d'uso principali;
- adeguatezza generale dell'esperienza utente.

0.6.2 Test backend

In questa sottosezione si possono riportare i test svolti o pianificati sui servizi backend, distinguendo se necessario tra:

- test unitari della logica applicativa;
- test di integrazione degli endpoint REST;
- verifica dei controlli di autorizzazione;
- verifica della correttezza dei payload in ingresso e in uscita.

Se una parte dei test non e' ancora stata completata, conviene dichiararlo in modo esplicito, indicando quali verifiche risultano gia' disponibili e quali rappresentano lavoro futuro.

0.6.3 Test frontend

Questa sottosezione puo' descrivere le verifiche effettuate sull'interfaccia, ad esempio:

- corretto rendering delle viste principali;
- aggiornamento dei dati a seguito delle azioni dell'utente;
- comportamento delle componenti di configurazione, chatbot e notifiche;
- fruibilita' su viewport differenti.

0.6.4 Test end-to-end dei flussi principali

E' utile riportare almeno alcuni scenari completi, come:

- registrazione o login e accesso alle funzionalita' protette;
- creazione di una configurazione e salvataggio;
- popolamento del carrello a partire da una configurazione;

- aggiornamento del catalogo da parte dell'amministratore e osservazione degli effetti sul sistema.

Per ciascuno scenario si possono indicare passi principali, risultato atteso e risultato osservato.

0.6.5 Valutazione qualitativa con utenti

Se sono stati svolti test informali o osservazioni su utenti reali o simulati, questa e' la sede giusta per riportarli. In particolare si possono discutere:

- facilità d'uso percepita del configuratore;
- comprensibilità del flusso di acquisto;
- utilità percepita di chatbot e notifiche;
- eventuali criticità emerse durante la prova.

0.6.6 Limiti dell'attività di test

In questa parte conviene esplicitare i limiti della verifica svolta, ad esempio presenza di test manuali predominanti, copertura non completa o parti ancora non implementate. Questa trasparenza rende il report più credibile e aiuta a contestualizzare i risultati.

0.6.7 Sintesi dei risultati

La sezione può chiudersi con una sintesi dei principali esiti della verifica, evidenziando quali obiettivi risultano già soddisfatti e quali miglioramenti restano pianificati per l'evoluzione del progetto.

0.7 Deployment

Questa sezione descrive come il sistema viene eseguito, configurato e avviato. L'obiettivo e' mostrare che l'architettura progettata può essere effettivamente messa in funzione in un ambiente coerente con le scelte tecnologiche effettuate.

0.7.1 Architettura di esecuzione

KompozeR prevede l'esecuzione coordinata di frontend, gateway, microservizi backend e basi di dati dedicate. In questa sottosezione e' utile descrivere come tali componenti vengano avviati e collegati tra loro in ambiente locale o di test.

Qui si può inserire un diagramma di deployment semplificato che mostri container o processi principali e relative dipendenze.

0.7.2 Containerizzazione dei servizi

Se il progetto utilizza Docker, questa parte dovrebbe spiegare come ogni componente applicativo venga pacchettizzato e reso eseguibile in modo consistente. Conviene chiarire:

- quali componenti vengono containerizzati;
- il ruolo delle immagini applicative;
- il vantaggio in termini di isolamento e riproducibilit .

0.7.3 Configurazione dell'ambiente

Per l'esecuzione del sistema   necessario definire variabili d'ambiente, porte di ascolto, URL dei servizi e parametri di connessione ai database. In questa sottosezione conviene riassumere la configurazione minima richiesta, evitando di riportare dati sensibili ma chiarendo la logica generale del setup.

0.7.4 Avvio del sistema in locale

Questa parte dovrebbe descrivere in modo sintetico la procedura necessaria per eseguire il progetto in ambiente di sviluppo o dimostrazione. Ad esempio:

- preparazione dell'ambiente;
- avvio dei servizi e dei database;
- avvio del frontend;
- verifica del corretto funzionamento iniziale.

Se utile, qui si possono richiamare i principali comandi o i file di configurazione che rendono possibile l'avvio coordinato del sistema.

0.7.5 Persistenza e dati iniziali

In questa sottosezione si pu  spiegare come vengono gestiti i dati minimi necessari all'avvio del sistema, ad esempio utenti amministrativi iniziali, prodotti di catalogo di esempio o configurazioni di base utili per test e dimostrazione.

0.7.6 Considerazioni operative

La sezione pu  chiudersi con alcune osservazioni sulle principali implicazioni operative della soluzione adottata: facilit  di setup, modularit  dell'ambiente, possibilit  di sostituzione o aggiornamento dei singoli servizi e supporto a una futura evoluzione del sistema.

0.8 Conclusioni

KompozeR e' stato concepito come un'estensione evoluta dell'e-commerce Kompo, con l'obiettivo di rendere piu' semplice e guidata la configurazione e l'acquisto di scaffalature personalizzabili. Il progetto affronta il problema sia dal punto di vista dell'esperienza utente, introducendo un configuratore visuale, un chatbot di supporto e notifiche contestuali, sia dal punto di vista dell'organizzazione software, tramite una struttura modulare basata su servizi distinti.

0.8.1 Risultati raggiunti

In questa sottosezione conviene riassumere i principali risultati del lavoro svolto, ad esempio:

- definizione dei requisiti e del dominio applicativo;
- progettazione di un'architettura web coerente con il problema affrontato;
- individuazione di servizi separati per autenticazione, catalogo, configurazione, carrello, notifiche, chatbot e reporting;
- definizione di contratti applicativi chiari per API, messaggi real-time e DTO di dominio.

0.8.2 Limiti attuali

E' utile esplicitare anche i limiti del progetto nella sua fase corrente, ad esempio eventuali funzionalita' solo progettate, copertura di test ancora parziale o assenza di alcune ottimizzazioni implementative. Una sezione di questo tipo rende il report piu' rigoroso e chiarisce bene il perimetro del lavoro effettivamente svolto.

0.8.3 Sviluppi futuri

Tra le evoluzioni naturali del progetto rientrano il consolidamento dell'implementazione, il completamento delle attivita' di verifica e il raffinamento delle funzionalita' piu' avanzate lato utente e lato amministrativo. In questa parte puoi anche richiamare il fatto che gli aspetti piu' strettamente distribuiti saranno approfonditi separatamente nel report DS.

0.8.4 Valutazione finale

Nel complesso, il progetto rappresenta un caso applicativo interessante per il contesto ASW perche' combina progettazione di servizi web, organizzazione modulare del backend, attenzione all'interazione utente e definizione accurata dei contratti tra componenti. Il report mette quindi in evidenza soprattutto la qualita' della soluzione come applicazione e servizio web, lasciando a un documento separato l'analisi dei temi distribuiti piu' avanzati.